📘 **FULL STACK DEVELOPMENT WITH MERN**

PROJECT DOCUMENTATION

---

1. INTRODUCTION

Project Title:

FitHub Web – Fitness Center Management System

Team Members:

- **Akshaya R** – Frontend Development (UI Design, React Components, Integration)
- **Gayathri S** – Backend Development (API Development, Database, Authentication)

---

2. PROJECT OVERVIEW

Purpose

The purpose of this project is to develop a web-based Fitness Center Management System using the MERN stack. The system aims to automate and manage the core operations of a fitness center, including membership management, class scheduling, trainer allocation, and booking processes.

Traditional fitness centers rely on manual processes that are inefficient and prone to errors. This project provides a digital solution that enhances efficiency, reduces workload, and improves user experience. The system ensures smooth interaction between members, trainers, and administrators through a centralized platform.

Features

- User Registration and Login System
- Membership Management
- Class Scheduling and Booking
- Trainer Management
- Admin Dashboard
- Secure Authentication System
- API Integration
- Responsive User Interface
- Notification System

---

3. ARCHITECTURE

Frontend (React.js)

The frontend of the application is developed using React.js, which follows a component-based architecture. The user interface is divided into reusable components that improve maintainability and scalability.

The application uses React Router for navigation between pages. State management is handled using hooks such as useState and useEffect. API calls are performed using Axios to communicate with the backend.

The frontend ensures a responsive and interactive user experience across different devices.

Backend (Node.js + Express.js)

The backend is implemented using Node.js and Express.js. It handles all business logic, API requests, authentication, and database communication.

The server follows a RESTful architecture and includes middleware for handling authentication, validation, and error management. The backend ensures secure and efficient processing of user requests.

Database (MongoDB)

MongoDB is used as the database for storing application data. It is a NoSQL database that stores data in a flexible JSON-like format.

Collections include:

- Users
- Trainers
- Classes
- Bookings

MongoDB provides scalability and fast data retrieval, making it suitable for modern web applications.

---

4. SETUP INSTRUCTIONS

Prerequisites

- Node.js installed
- MongoDB installed or cloud database
- npm or yarn
- Git

Installation Steps

1. Clone the repository

```
git clone https://github.com/your-repo/fithub-web.git
```

2. Navigate to project directory

```
cd fithub-web
```

3. Install frontend dependencies

```
cd client
npm install
```

4. Install backend dependencies

```
cd ../server
npm install
```

Environment Variables

Create a .env file in the server folder:

```
MONGO_URI=your_mongodb_connection
JWT_SECRET=your_secret_key
PORT=5000
```

---

5. FOLDER STRUCTURE

Client

```
client/
 ├── src/
 │   ├── components/
 │   ├── pages/
 │   ├── services/
 │   ├── App.js
 │   └── index.js
```

Server

```
server/
```

```
├── models/
├── routes/
├── controllers/
├── middleware/
├── config/
├── server.js
```

---

6. RUNNING THE APPLICATION

Frontend

cd client

npm start

Runs on: http://localhost:3000

Backend

cd server

npm start

Runs on: http://localhost:5000

---

7. API DOCUMENTATION

User APIs

Register User
POST /api/users/register

Request:

```
{
  "name": "Akshaya",
  "email": "akshaya@gmail.com",
  "password": "123456"
}
```

Login User
POST /api/users/login

Class APIs

GET /api/classes
POST /api/bookings

Trainer APIs

GET /api/trainers
POST /api/trainers

---

8. AUTHENTICATION

Authentication is implemented using JSON Web Tokens (JWT). When a user logs in, a token is generated and sent to the client. This token is stored and used for accessing protected routes.

The backend verifies the token using middleware before allowing access to secure endpoints. This ensures data security and prevents unauthorized access.

---

9. USER INTERFACE

The user interface is designed to be simple, intuitive, and responsive. It includes pages such as login, registration, dashboard, class booking, and admin panel.

The UI provides smooth navigation and enhances user experience through clear layouts and interactive elements.

---

10. TESTING

Testing is performed to ensure that the system works correctly under different conditions.

Types of testing include:

- Functional Testing
- API Testing
- Integration Testing

Tools such as Postman are used to test API endpoints and verify responses.

---

11. SCREENSHOTS / DEMO

Screenshots of the application include:

- Login Page
- Dashboard

- Booking System
- Admin Panel

These demonstrate the functionality and design of the system.

---

12. KNOWN ISSUES

- Limited UI customization in some components
- Performance may vary with large datasets
- Some features may require further optimization

---

13. FUTURE ENHANCEMENTS

- Integration with mobile applications
- Online payment gateway
- AI-based recommendations
- Real-time notifications
- Advanced analytics dashboard